Valid years can still overflow savings

12 October 2026 | C003a consolidation and readiness | Synthetic fixtures only

Sidiq review status: Not yet reviewed. Studied October 12; integrated October 13.

Research question and hypothesis

Does accepting whole-year text guarantee a usable savings result? Hypothesis: no. Blank, fraction, suffix and oversized text stop before calculation, while the largest accepted integer can reach calculation and overflow. Valid zero is different from both kinds of failure.

This consolidates the existing functions after reproduction; the ledger starts with a later hand example.

Beginner explanation and terms

A contract states which inputs and results a function permits. Parsing checks the original spelling and whether its number fits an int. Calculation checks principal, rate, years and the finite result. An exception reports that a normal result could not be supplied. A catch handles a particular error; a return exits this trace before later statements execute. It must not manufacture a zero-year value.

Think of two separate questions. First, did the user supply an allowed whole count? Second, can this model compute a permitted finite balance using that count? Passing the first does not answer the second. The local variable starts at zero only to initialise storage. The error branches exit before using it.

Assumptions, mathematics and hand example

P is principal in arbitrary money units; r is the fixed effective annual rate as a fraction; n is a nonnegative whole count of model years. B(n) = P * (1+r)^n. Here P=1000 and normally r=0.05. B(0)=1000; B(1)=1000+50=1050; B(2)=1050+52.50=1102.50. Python Decimal independently checks B(2).

At the tested int maximum, n=2147483647 is valid text and a valid integer. With r=0.05, the finite double calculation fails. We need no enormous decimal balance to predict that failure: 1.05^20 > 2, so after 20480 years the growth factor exceeds 2^1024, beyond the tested binary64 finite range. The chosen horizon is far larger. This is a magnitude bound, not a measured financial loss.

With the same maximum n but r=0, B(n)=1000. The existing zero-rate branch returns principal directly. This positive control shows why rejecting every large year count in the parser would change its contract. The observed overflow describes this implementation and target, not every numeric representation.

Fixed case	Parser result	Calculation result
Blank	Spelling error	Skipped
0	0 years	1000.00
002	2 years	1102.50
2.5	Spelling error	Skipped
2x	Spelling error	Skipped
Maximum text with an appended 0	Range error	Skipped
Maximum at 5%	2147483647 years	Overflow; no balance
Maximum at zero rate	2147483647 years	1000.00

Trace the C++ branches slowly

The fixed readiness_trace.cpp calls existing functions. Its helper accepts a label, original text and rate. A const string reference lets it inspect text without changing it. int years = 0 initialises a local variable; years = parse_whole_years(text) assigns it only if parsing returns normally.

The first try contains only parsing. invalid_argument means forbidden spelling, and out_of_range means digit-only text beyond the representation. Each matching catch prints the reason and returns. Thus neither error reaches compound_balance. Blank is not reinterpreted as zero.

After successful parsing, the helper prints the accepted integer. Its second try calls compound_balance(1000.0, rate, years). A finite result is printed to two decimals. overflow_error prints no balance. Unexpected exceptions are not hidden behind a generic successful outcome.

The main function makes eight explicit calls. It uses the already introduced numeric_limits to obtain the target maximum. No input loop, vector, new class or CLI is added. Production sources stay unchanged.

Appending a decimal zero multiplies the represented positive maximum by ten in text; no C++ integer overflow is used to form the out-of-range input. A fraction still has no defined accrual convention here.

Protocol, implementation and observed checks

Before running, the table above predicts the eight outcomes. The Python verifier freshly compiles one C++17 executable with -Wall -Wextra -Wpedantic -Werror in an empty temporary directory. It runs it twice, requiring exact stdout, empty stderr and exit 0. Temporary binaries are cleaned on exit.

Run .venv/bin/python scripts/verify_years_readiness.py from the repository root, without -O. The recipe is intentionally tied to the observed target maximum in the archived oracle and refuses a changed value. It independently checks the two-year Decimal result and growth-factor magnitude bound. experiments/years_parser/readiness-2026-10-12.json records the output, source hashes, compiler and current published baseline.

Observed: both runs match all eight predictions; four parser rejections, three finite balances and one accepted-then-overflow case. Apple Clang 21 compiled without diagnostics. The experiment passed on its first run. This finite trace consolidates branch behaviour; it is not an exhaustive test suite.

Shared validation earlier in today's integration passed make check: 608 Python tests, Ruff lint and 52 formatted files, strict mypy on 21 package files, 66 savings and 30 parser checks, 75 boundary oracle comparisons, four golden outputs and fractional controls. pip consistency passed. Standard Make binaries were reused; the reproduction recipe separately freshly compiled six executables.

Costs, failure cases and limitations

The savings model assumes a fixed nonnegative rate, annual reinvestment and no intervening cash flows. Fees, taxes and inflation are omitted; zero observed costs are not inferred. No traded instrument, historical period, market sample, spread estimate, tuning or profitability result is present.

A fallback zero hides malformed input. A broad catch could confuse parsing and calculation errors. Formatting two decimals is not exact-money arithmetic. This implementation can also reject an overflowing intermediate factor even when another numerical method could produce a finite final value.

No length limit, external CLI hardening, cross-platform guarantee, all-input proof or coverage percentage is established. Python and C++ share the written contract and fixture expectations. The current checks support repository readiness for the next bounded lesson; they do not establish personal understanding.

Evidence and readiness decision

C003a's repository teaching slice is complete: contract, implementation, independent boundary checks, deliberate failure, fresh reproduction and consolidation each have evidence. Saved locally October 12, this trace was integrated October 13 after eight input and seven artifact hashes matched. The standard make check now runs a fresh temporary build and both exact traces; production functions are unchanged.

C003b can be scheduled later, but is not complete: principal/rate spelling, finite-value policy, units, external input limits and full CLI integration still need separate lessons. C003 remains in progress. C004 returns/vectors and C005 statistics are also unfinished. No personal review transition is inferred.

Proceed next with L001a: cash, position and marked wealth using one hand calculation. It needs no CLI. Introduce every new struct or timing concept before implementation. L002 delayed execution and its future-price invariance test remain separate. Trend dates depend on accounting, timing and returns; shift later work rather than compressing lessons. Q004a/Q005b still gate empirical research.

Publication, progress and necessary input

Today's shared draw is 2, and the actual count is 2. Commit 609225c integrates October 11 reproduction; c8c925e integrates the October 11 weekly synthesis and reviewed backlog refinement. Both were pushed; independent remote main lookup matched c8c925e04842f65827dac96def46f107df9b3586. CI run 38245092052 passed for that exact head. This lesson was not included in that CI result or those commits.

No input blocks synthetic continuation. Any uncertainty about these branches can trigger repetition; only Sidiq can confirm personal review. Lawful source/access evidence is needed before real acquisition. The daily schedule remains 08:00 Europe/London including daylight-saving changes. Stop new research after December 31, 2026 and request the next scope. October 13 publication uses its actual date and shared draw of 3; the study date stays October 12.

Primary sources consulted on October 12

Microsoft charconv reference: the conversion reports a stopping position and an error code; the project adds its explicit digit-only spelling contract before using the returned integer.
https://learn.microsoft.com/en-us/cpp/standard-library/charconv-functions?view=msvc-170

Microsoft Standard conversions: fractional-to-integer conversion discards the fraction within its representable domain. Casting a prior floating value cannot validate its original spelling.
https://learn.microsoft.com/en-us/cpp/cpp/standard-conversions?view=msvc-170

Python subprocess reference: capture output, check return status and bound execution with a timeout.
https://docs.python.org/3/library/subprocess.html#subprocess.run

Interview explanation

I preserve original text until parsing checks spelling and range. A rejected input never reaches the calculation. A valid integer can still overflow the financial model, so that model validates its own result. Separate error paths explain which contract failed without silently substituting zero.

Check your understanding

1. Why does initialising years to zero not make blank input a valid zero-year request?

2. Why does the maximum integer pass parsing but fail at a positive rate?

3. Why does the same maximum at zero rate produce a balance?

4. Does this readiness record establish Sidiq's mastery or a complete CLI?

Suggested answers

1. The parser throws on blank input. Its catch returns before the local value can enter the model. The explicit string 0 instead passes parsing and gives a permitted horizon.

2. Integer representation and finite exponential growth are different constraints. The text fits int, but the growth factor exceeds the tested double range; the model rejects it rather than returning infinity.

3. Zero rate means no growth, so the existing shortcut returns the validated principal. This is a permitted calculation, whereas rejection supplies no result at all.

4. No. Repository evidence supports the next bounded lesson. Personal review is unconfirmed, and principal/rate parsing, resource policy and the complete CLI remain future work.